CI-0123

Proyecto Integrador de Redes y Sistemas Operativos

Protocolo de comunicación

Subgrupo 5-Grupo 3,2026
Estudiantes: Kiara Virginia Brenes Díaz
Carlos Alavarado
Universidad de Costa Rica - Escuela de Ciencias de la Computacion e Informatica

1. introducción

Este documento especifica el protocolo de comunicación diseñado para el proyecto integrador CI-0123. El sistema permite que múltiples clientes obtengan las piezas necesarias para armar figuras de Lego a través de una arquitectura distribuida compuesta por clientes, servidores intermedios ("tenedores") y servidores de piezas. El protocolo define con precisión como se estructuran los mensajes, que comandos existen en cada capa de comunicación, como se maneja la concurrencia y que ocurre cuando un servidor local no puede satisfacer una solicitud. El diseño es modular: se pueden agregar nuevos comandos en el futuro sin alterar la estructura base.

2. Arquitectura del Sistema

El sistema se organiza en torno al concepto de isla. Cada isla agrupa tres componentes con roles bien diferenciados:

- Cliente: punto de entrada del usuario. Construye los mensajes del protocolo, los envia al Tenedor y presenta la respuesta al usuario. No se comunica directamente con el Servidor de Piezas ni con los componentes de otras islas.
- Tenedor (Servidor Intermedio): nucleo de enrutamiento. Valida mensajes entrantes, administra una cola FIFO de solicitudes, delega trabajo a un pool de hilos trabajadores y decide si resolver la solicitud localmente o consultar islas remotas. Mantiene una tabla de enrutamiento con las direcciones de los Tenedores conocidos.
- Servidor de Piezas: almacenamiento de la isla. Recibe comandos exclusivamente del Tenedor local, busca piezas entre sus recursos y responde con los datos encontrados o con un error. Aplica exclusion mutua a nivel de recurso para soportar acceso concurrente.

El flujo general de comunicacion sigue este esquema:

```
Cliente → Tenedor → Servidor de Piezas
Cliente ← Tenedor ← Servidor de Piezas
```

Cuando el Servidor de Piezas local no tiene la pieza, el Tenedor consulta islas remotas:

```
Tenedor A → Tenedor B → Servidor de Piezas de B
Tenedor A ← Tenedor B ← Servidor de Piezas de B
```

3. Convenciones del Protocolo

3.1 Formato General de un Mensaje

Todos los mensajes del sistema siguen la misma estructura de texto plano:

```
COMANDO <parametro1> <parametro2> ... <parametroN>
```

Reglas generales:

- Los comandos y parametros fijos se escriben unicamente en mayuscula.
- Los nombres de piezas deben escribirse exactamente igual a como fueron registrados en el Servidor de Piezas.
- Los parametros se separan por un espacio simple.
- Los mensajes de error incluyen un codigo de error seguido de una descripcion textual.

3.2 Tipos de Parametros

Parametro	Descripcion	Ejemplo
<nombre_pieza>	Identificador unico de la pieza o figura solicitada.	brick_2x4_rojo
<datos_pieza>	Contenido de la pieza codificado en Base64.	GvRWcaqB==
<id_solicitud>	UUID v4 generado por el Tenedor para rastrear cada solicitud.	a5f6-98bc-...
<dir_isla>	Direccion IP y puerto del Tenedor remoto.	162.198.1.5:7070
<cliente_id>	Identificador entero unico asignado al cliente en su sesion.	15
<nombre_error>	Codigo de error en mayuscula (ver seccion 12).	NO_ENCONTRADA
<descripcion>	Texto libre que explica el resultado o el error.	No encontrado en esta isla

4. Comandos de Sesion — Cliente a Tenedor

Estos comandos gestionan el ciclo de vida de la conexion del cliente con el sistema.

4.1 CONNECT

Proposito: establecer una sesion activa entre el cliente y el Tenedor. Debe enviarse antes de cualquier otra solicitud.

```
CONNECT <cliente_id>
```

Respuesta esperada:

```
OK
```

4.2 DISCONNECT

Proposito: cerrar la sesion del cliente de forma ordenada.

```
DISCONNECT <cliente_id>
```

Respuesta esperada:

```
OK
```

4.3 PING

Proposito: verificar que el Tenedor esta activo y responde correctamente.

```
PING
```

Respuesta esperada:

```
PONG
```

4.4 STATUS

Proposito: solicitar informacion sobre el estado actual del Tenedor (cola, hilos activos, islas conocidas).

```
STATUS
```

Respuesta esperada:

```
OK STATUS <descripcion>
```

5. Comandos Intra-isla — Cliente a Tenedor

Estos comandos permiten al usuario interactuar con el inventario de piezas disponible en el sistema. El Cliente es el unico emisor; el Tenedor es el unico receptor en esta capa.

5.1 LISTAR

Proposito: solicitar el listado completo de piezas disponibles en la isla local.

```
LISTAR
```

Respuesta esperada del Tenedor:

```
OK LISTAR <nombre_p1> <nombre_p2> ... <nombre_pN>
OK LISTAR VACIO     // si no hay piezas
ERROR <nombre_error> <descripcion>  // si ocurre un fallo interno
```

> *Nota: el listado refleja unicamente las piezas del Servidor de Piezas local. No incluye las de islas remotas.*

5.2 SOLICITAR

Proposito: pedir al sistema que entregue una pieza especifica por su nombre.

```
SOLICITAR <nombre_pieza>
```

Respuesta del Tenedor (dos momentos):

```
OK SOLICITAR                          // acuse de recibo inmediato
REENVIAR <nombre_pieza> <datos_pieza>  // cuando el Tenedor resuelve la consulta
```

```
ERROR <nombre_pieza> NO_ENCONTRADA      // si no existe en ninguna isla conocida
```

El OK inicial confirma que la solicitud fue recibida y encolada. El REENVIAR llega en un segundo momento. El Tenedor valida el formato del mensaje antes de responder OK; si falla, responde ERROR FORMATO_INVALIDO sin encolar.

5.3 GET

Proposito: solicitar acceso directo a un recurso especifico, util para consultar figuras completas.

```
GET <recurso>
```

Respuesta esperada:

```
OK <datos>
NOTFOUND  // si el recurso no existe
```

5.4 READ

Proposito: solicitar un elemento especifico dentro de un recurso.

```
READ <recurso> <indice>
```

Respuesta esperada:

```
DATA <valor>
```

5.5 SEARCH

Proposito: buscar un elemento especifico dentro de un recurso por su nombre o descripcion.

```
SEARCH <recurso> <valor>
```

Respuesta esperada:

```
FOUND <indice>
NOTFOUND  // si no se encuentra
```

6. Comandos Intra-isla — Tenedor a Servidor de Piezas

Esta capa cubre los mensajes que circulan internamente dentro de una isla entre el Tenedor y su Servidor de Piezas. El Cliente no participa en estos intercambios.

6.1 BUSCAR

Proposito: instruir al Servidor de Piezas para que busque una pieza en sus recursos locales.

Emisor: Hilo trabajador del Tenedor. Receptor: Servidor de Piezas local.

```
BUSCAR <nombre_pieza> <id_solicitud>
```

Respuestas posibles del Servidor de Piezas:

```
ENTREGAR <nombre_pieza> <datos_pieza> <id_solicitud>  // pieza encontrada
ERROR <nombre_pieza> NO_ENCONTRADA <id_solicitud>    // pieza no encontrada
OCUPADO <nombre_pieza> <id_solicitud>                // recurso bloqueado
```

Si el Servidor responde OCUPADO, el hilo trabajador reencola la solicitud para reintentarla mas adelante.

6.2 ENTREGAR

Proposito: respuesta del Servidor de Piezas con el contenido de la pieza encontrada. Emisor: Servidor de Piezas. Receptor: Tenedor.

```
ENTREGAR <nombre_pieza> <datos_pieza> <id_solicitud>
```

6.3 REENVIAR

Proposito: el Tenedor entrega al Cliente la pieza ya resuelta. Emisor: Tenedor. Receptor: Cliente.

```
REENVIAR <nombre_pieza> <datos_pieza>
OK REENVIAR <nombre_pieza>  // confirmacion del Cliente
```

6.4 REGISTER

Proposito: permite que un Servidor de Piezas se registre en su Tenedor local, anunciando su presencia y el inventario de figuras que custodia. Emisor: Servidor de Piezas. Receptor: Tenedor.

```
REGISTER <dir_servidor> <figura1> <figura2> ... <figuraN>
```

Respuesta esperada:

```
OK REGISTER
```

Al recibir este mensaje, el Tenedor actualiza su tabla de rutas y notifica a los demas Tenedores conocidos mediante NOTIFY_REGISTER.

6.5 UNREGISTER

Proposito: el Servidor de Piezas notifica al Tenedor que esta abandonando el sistema.

```
UNREGISTER <dir_servidor>
```

Respuesta esperada:

```
OK UNREGISTER
```

El Tenedor actualiza su tabla de rutas y notifica a los demas Tenedores mediante NOTIFY_UNREGISTER.

7. Comandos Inter-isla — Tenedor a Tenedor

Cuando el Servidor de Piezas local no tiene una pieza, el Tenedor consulta los Tenedores de otras islas. Esta comunicacion usa el mismo canal TCP que el resto del protocolo pero emplea comandos exclusivos.

7.1 CONSULTAR_ISLA

Proposito: preguntar a un Tenedor remoto si su isla posee la pieza buscada. Solo se emite cuando el Servidor local ya respondio ERROR NO_ENCONTRADA.

Emisor: Tenedor A (local). Receptor: Tenedor B (remoto).

```
CONSULTAR_ISLA <nombre_pieza> <id_solicitud> <dir_isla_origen>
```

El parametro dir_isla_origen identifica la direccion del Tenedor A para que B sepa a quien responder.

Comportamiento del Tenedor B al recibir CONSULTAR_ISLA:

- Trata la consulta exactamente igual que una solicitud interna: encola la peticion y despacha un hilo trabajador que envia BUSCAR a su Servidor de Piezas local.
- Si su Servidor responde ENTREGAR, el Tenedor B responde con REMITIR hacia el Tenedor A.
- Si su Servidor responde ERROR NO_ENCONTRADA, el Tenedor B responde con NO_DISPONIBLE hacia el Tenedor A.
- El Tenedor B nunca consulta a otras islas en nombre del Tenedor A.

Respuestas posibles del Tenedor B:

```
REMITIR <nombre_pieza> <datos_pieza> <id_solicitud>  // la isla B tiene la pieza
NO_DISPONIBLE <nombre_pieza> <id_solicitud>         // la isla B no tiene la pieza
```

7.2 REMITIR

Proposito: enviar la pieza encontrada en la isla remota hacia la isla consultante. Es la respuesta afirmativa a CONSULTAR_ISLA. Emisor: Tenedor B. Receptor: Tenedor A.

```
REMITIR <nombre_pieza> <datos_pieza> <id_solicitud>
```

Al recibir REMITIR, el Tenedor A identifica la solicitud original por id_solicitud y procede a enviar REENVIAR al Cliente.

7.3 NO_DISPONIBLE

Proposito: indicar que la isla remota no tiene la pieza solicitada. Es la respuesta negativa a CONSULTAR_ISLA. Emisor: Tenedor B. Receptor: Tenedor A.

```
NO_DISPONIBLE <nombre_pieza> <id_solicitud>
```

El Tenedor A consulta la siguiente isla en su tabla de enrutamiento. Si agota todas las islas conocidas sin obtener REMITIR, responde al Cliente con ERROR NO_ENCONTRADA.

7.4 NOTIFY_REGISTER

Proposito: propagar a todos los Tenedores activos el registro de un nuevo Servidor de Piezas. Se emite automaticamente tras procesar un REGISTER exitoso. Emisor: Tenedor A. Receptor: Todos los demas Tenedores conocidos.

```
NOTIFY_REGISTER <dir_servidor_piezas> <figura1> <figura2> ... <figuraN>
```

Respuesta esperada de cada Tenedor receptor:

```
OK NOTIFY_REGISTER
```

7.5 NOTIFY_UNREGISTER

Proposito: notificar a todos los Tenedores activos que un Servidor de Piezas ha dejado de funcionar o se ha desconectado. Emisor: Tenedor A. Receptor: Todos los demas Tenedores conocidos.

```
NOTIFY_UNREGISTER <dir_servidor_piezas>
```

Respuesta esperada:

```
OK NOTIFY_UNREGISTER
```

7.6 SYNC_ROUTES

Proposito: cuando un Tenedor recien iniciado necesita obtener el estado actualizado de todos los Servidores de Piezas existentes en el sistema.

```
SYNC_ROUTES <dir_isla_solicitante>
```

Respuesta esperada:

```
OK SYNC_ROUTES <dir_s1> <fig1a> <fig1b> | <dir_s2> <fig2a> | ...
```

8. Comandos de Concurrencia

Estos comandos no viajan por la red; son instrucciones internas que los hilos trabajadores intercambian con el mecanismo de control de acceso del Servidor de Piezas.

Comando	Emisor	Receptor	Descripcion
BLOQUEAR	Hilo trabajador	Mutex del recurso	Solicitar acceso exclusivo a una pieza antes de leerla o modificarla.
LIBERAR	Hilo trabajador	Mutex del recurso	Liberar el acceso a la pieza una vez finalizada la operación.
OCUPADO	Mutex del recurso	Hilo trabajador	El recurso ya esta bloqueado; el hilo debe esperar o reencolar la solicitud.

9. Fases del Protocolo

Una solicitud completa de pieza atraviesa cuatro fases numeradas. La Fase 3b es opcional y solo se activa si el Servidor de Piezas local no resuelve la solicitud.

Fase 1 — Solicitud del Cliente

El usuario indica que pieza o figura quiere. El Cliente construye el mensaje (CONNECT si aun no tiene sesion, luego SOLICITAR) y lo envia al Tenedor. El Tenedor valida el formato y responde OK SOLICITAR como acuse de recibo, o ERROR FORMATO_INVALIDO si el mensaje es invalido.

Fase 2 — Encolado y Despacho

Esta fase es invisible para el Cliente. El Tenedor coloca la solicitud validada en su cola FIFO interna (ENCOLAR). El hilo receptor adquiere el mutex de la cola, inserta la solicitud al final y ejecuta signal() para despertar a un hilo trabajador durmiente. Un hilo libre extrae la solicitud (DESPACHAR) y la entrega al Servidor de Piezas mediante BUSCAR.

Fase 3a — búsqueda Local

El Servidor de Piezas recibe BUSCAR, adquiere el mutex del recurso solicitado (BLOQUEAR), realiza la busqueda y libera el mutex (LIBERAR). Si encuentra la pieza, responde ENTREGAR; si no, responde ERROR NO_ENCONTRADA.

Fase 3b — Consulta Inter-isla (Condicional)

Solo se ejecuta si la Fase 3a termina con ERROR NO_ENCONTRADA. El Tenedor itera sobre su tabla de enrutamiento enviando CONSULTAR_ISLA a cada Tenedor remoto conocido, en orden, hasta recibir REMITIR o agotar la lista.

Fase 4 — Respuesta al Cliente

Una vez que el Tenedor tiene la pieza (de su propio Servidor o de una isla remota), la envia al Cliente con REENVIAR. El Cliente confirma la recepcion con OK REENVIAR. Si ninguna isla pudo resolver la solicitud, el Tenedor responde ERROR NO_ENCONTRADA al Cliente.

10. Control de Concurrencia

El sistema puede recibir múltiples solicitudes simultaneas. Se emplean tres mecanismos para garantizar que ninguna solicitud se pierda ni que los datos se corrompan.

10.1 Mutex de la Cola del Tenedor

La cola FIFO del Tenedor es un recurso compartido entre el hilo receptor y todos los hilos trabajadores. Toda operacion de insercion (ENCOLAR) o extraccion (DESPACHAR) requiere adquirir el mutex de la cola. Esto garantiza que solo un hilo accede a la estructura de la cola en cada instante.

10.2 Variable de condición

Los hilos trabajadores no hacen busy-waiting sobre la cola. Se bloquean en una variable de condicion. Cuando el hilo receptor inserta una solicitud, ejecuta signal() sobre esa variable y despierta exactamente un hilo trabajador. Esto elimina el desperdicio de CPU cuando no hay solicitudes pendientes.

10.3 Mutex por Recurso en el Servidor de Piezas

Cada pieza almacenada en el Servidor tiene su propio mutex independiente. Dos hilos que soliciten piezas distintas pueden operar en paralelo sin bloquearse entre si. Esta granularidad fina maximiza el paralelismo real del sistema.

11. Tabla de Comandos

Comando	Emisor	Receptor	Capa
CONNECT	Cliente	Tenedor	sesión
DISCONNECT	Cliente	Tenedor	sesión
PING	Cliente	Tenedor	sesión
STATUS	Cliente	Tenedor	sesión
LISTAR	Cliente	Tenedor	Intra-isla
SOLICITAR	Cliente	Tenedor	Intra-isla
GET	Cliente	Tenedor	Intra-isla
READ	Cliente	Tenedor	Intra-isla
SEARCH	Cliente	Tenedor	Intra-isla
BUSCAR	Tenedor (hilo)	Servidor Piezas	Intra-isla
ENTREGAR	Servidor Piezas	Tenedor	Intra-isla

Comando	Emisor	Receptor	Capa
REENVIAR	Tenedor	Cliente	Intra-isla
REGISTER	Servidor Piezas	Tenedor	Registro
UNREGISTER	Servidor Piezas	Tenedor	Registro
CONSULTAR_ISLA	Tenedor A	Tenedor B	Inter-isla
REMITIR	Tenedor B	Tenedor A	Inter-isla
NO_DISPONIBLE	Tenedor B	Tenedor A	Inter-isla
NOTIFY_REGISTER	Tenedor A	Tenedores	Inter-isla
NOTIFY_UNREGISTER	Tenedor A	Tenedores	Inter-isla
SYNC_ROUTES	Tenedor nuevo	Tenedor activo	Inter-isla
BLOQUEAR	Hilo trabajador	Mutex recurso	Concurrencia
LIBERAR	Hilo trabajador	Mutex recurso	Concurrencia
OCUPADO	Mutex recurso	Hilo trabajador	Concurrencia
OK	Cualquier receptor	Emisor anterior	Global
ERROR	Cualquier receptor	Emisor anterior	Global
PONG	Tenedor	Cliente	Global
NOTFOUND	Tenedor	Cliente	Global
DATA	Tenedor	Cliente	Global
FOUND	Tenedor	Cliente	Global

12. Codigos de Error

Codigo	Descripcion	Emisor
FORMATO_INVALIDO	El mensaje no sigue la estructura del protocolo.	Tenedor
COMANDO_DESCONOCIDO	El comando enviado no existe en el protocolo.	Tenedor
NO_ENCONTRADA	La pieza no existe en ninguna isla conocida.	Tenedor
TIEMPO_AGOTADO	La isla remota no respondió dentro del tiempo límite.	Tenedor A
SERVIDOR_NO_DISPONIBLE	El Servidor de Piezas local no puede atender la solicitud.	Servidor Piezas
ERROR_INTERNO	Fallo inesperado en cualquier componente del sistema.	Cualquiera

13. Ejemplo Completo de Interaccion

A continuacion se muestra una secuencia de mensajes completa para ilustrar el protocolo en funcionamiento:

```
Cliente  → CONNECT 12
Tenedor  → OK
Cliente  → LISTAR
Tenedor  → OK LISTAR brick_1x1 brick_2x4 placa_2x2

Cliente  → SOLICITAR brick_2x4
Tenedor  → OK SOLICITAR
  [Tenedor encola, hilo trabajador envia BUSCAR al Servidor de Piezas]
  Hilo     → BUSCAR brick_2x4 a5f6-98bc
  Servidor → ENTREGAR brick_2x4 GvRWcaqB== a5f6-98bc
Tenedor  → REENVIAR brick_2x4 GvRWcaqB==
Cliente  → OK REENVIAR brick_2x4

Cliente  → SOLICITAR pieza_rara
Tenedor  → OK SOLICITAR
  [Servidor local responde ERROR NO_ENCONTRADA — se activa Fase 3b]
  Tenedor A → CONSULTAR_ISLA pieza_rara b7c1-12de 162.198.1.5:7070
  Tenedor B → REMITIR pieza_rara XyZ123== b7c1-12de
Tenedor A → REENVIAR pieza_rara XyZ123==
Cliente  → OK REENVIAR pieza_rara

Cliente  → DISCONNECT 12
Tenedor  → OK
```

14. Bitacoras (Logging)

Durante el desarrollo nos dimos cuenta de que necesitabamos llevar un registro de lo que pasa en el sistema. Sin esto, cuando algo falla es practicamente imposible saber que ocurrio. Tambien los profesores piden explicitamente que cada servidor pueda consultar sus bitacoras en linea, asi que definimos que eventos vale la pena guardar.

Cada componente (cliente, tenedor, servidor de piezas) escribe en su propio archivo de bitacora. Los archivos se llaman bitacora_cliente.log, bitacora_tenedor.log y bitacora_servidor.log respectivamente. Cada linea sigue este formato:

```
[YYYY-MM-DD HH:MM:SS] EVENTO descripcion
```

La descripcion incluye los parametros relevantes del evento, como identificadores, direcciones IP o nombres de piezas.

14.1 Eventos que registramos

Evento	Descripcion	Ejemplo
CONEXION	Un cliente se conecta al tenedor.	[2026-03-24 14:02:11] CONEXION cliente_id=12 desde=10.0.0.5
DESCONEXION	Un cliente cierra su sesión.	[2026-03-24 14:15:44] DESCONEXION cliente_id=12
SOLICITUD	El tenedor recibe y encola una solicitud de pieza.	[2026-03-24 14:02:15] SOLICITUD pieza=brick_2x4 id=a5f6-98bc
BUSQUEDA_LOCAL	El Servidor de Piezas recibe un BUSCAR.	[2026-03-24 14:02:16] BUSQUEDA_LOCAL pieza=brick_2x4 id=a5f6-98bc

Evento	Descripcion	Ejemplo
ENTREGA	El Servidor de Piezas encontro la pieza y responde ENTREGAR.	[2026-03-24 14:02:16] ENTREGA pieza=brick_2x4 id=a5f6-98bc
REENVIO	El tenedor envia la pieza resuelta al cliente.	[2026-03-24 14:02:17] REENVIO pieza=brick_2x4 cliente_id=12
CONSULTA_ISLA	El tenedor consulta a otro tenedor remoto.	[2026-03-24 14:03:01] CONSULTA_ISLA pieza=pieza_rara destino=162.198.1.5:7070
REMISION	Un tenedor remoto responde con REMITIR.	[2026-03-24 14:03:02] REMISION pieza=pieza_rara desde=162.198.1.5:7070
NO_DISPONIBLE	Un tenedor remoto responde que no tiene la pieza.	[2026-03-24 14:03:02] NO_DISPONIBLE pieza=pieza_rara desde=162.198.1.5:7070
ERROR	Cualquier error en cualquier componente.	[2026-03-24 14:05:00] ERROR tipo=NO_ENCONTRADA pieza=pieza_x
REGISTRO	Un Servidor de Piezas se registra en el tenedor.	[2026-03-24 08:00:01] REGISTRO servidor=10.0.0.3:8080 figuras=figura_A figura_B
BAJA	Un Servidor de Piezas se desregistra del tenedor.	[2026-03-24 18:00:00] BAJA servidor=10.0.0.3:8080
SYNC	Un tenedor sincroniza su tabla de rutas con otro.	[2026-03-24 08:00:05] SYNC solicitante=10.0.0.7:7070
OCUPADO	Un recurso estaba bloqueado; la solicitud se reencoló.	[2026-03-24 14:02:20] OCUPADO pieza=brick_2x4 id=a5f6-98bc reintentos=1
PING	Un cliente envía PING al tenedor.	[2026-03-24 14:01:00] PING cliente_id=12
STATUS	Un cliente solicita el estado del tenedor.	[2026-03-24 14:01:05] STATUS cliente_id=12 cola=3 hilos_activos=2

14.2 Formato detallado

Todos los campos de la linea de bitacora son obligatorios. El timestamp usa la zona horaria local del servidor. El nombre del evento se escribe en mayusculas. La descripcion usa pares clave=valor separados por espacios para facilitar el parsing automatico.

```
# Formato general:
[YYYY-MM-DD HH:MM:SS] EVENTO clave1=valor1 clave2=valor2 ...

# Ejemplo de secuencia completa en bitacora_tenedor.log:
[2026-03-24 14:02:11] CONEXION       cliente_id=12 desde=10.0.0.5
[2026-03-24 14:02:15] SOLICITUD      pieza=brick_2x4 id=a5f6-98bc cliente_id=12
[2026-03-24 14:02:17] REENVIO        pieza=brick_2x4 id=a5f6-98bc cliente_id=12
[2026-03-24 14:03:01] CONSULTA_ISLA  pieza=pieza_rara id=b7c1-12de destino=162.198.1.5:7070
[2026-03-24 14:03:02] REMISION       pieza=pieza_rara id=b7c1-12de desde=162.198.1.5:7070
[2026-03-24 14:03:02] REENVIO        pieza=pieza_rara id=b7c1-12de cliente_id=12
[2026-03-24 14:15:44] DESCONEXION    cliente_id=12
```

14.3 Consulta de bitacoras en linea

Cada componente expone un sub-comando STATUS LOG que permite consultar las ultimas N lineas de la bitacora sin detener el servicio:

```
STATUS LOG <N>

# Respuesta:
OK LOG <linea_1>
OK LOG <linea_2>
...
OK LOG END
```

El parametro N indica cuantas líneas devolver (máximo 100). Si N se omite, se devuelven las ultimas 20 lineas por defecto. El Servidor de Piezas soporta el mismo sub-comando, respondiendo desde su bitacora_servidor.log.